

ניתוח תעבורה בפרוטוקול TCP/IP

דוח מסכם: חלק 1 - אריזה ולכידת מנות

א. יצירת קובץ CSV עם הודעות בשכבת היישום

בשלב הראשון בנינו את בסיס הנתונים של הפרויקט. יצרנו קובץ CSV המכיל הודעות המדמות תעבורת רשת אמיתית בשכבת היישום (Application Layer).

- **תיאור:** הקובץ כולל עמודות עבור סוג הפרוטוקול, כמו "HTTP" כתובות מקור ויעד, פורטים, כמו 80 עבור "HTTP" ותוכן ההודעה עצמה.
- דוגמה להודעה:

[PROJECT-GROUP-01] GET /index.html HTTP/1.1

"איור 1: טעינת נתוני שכבת היישום מקובץ CSV והצגתם בטבלת DataFrame"

```
[2]: #TODO: Load CSV file with messages into pandas DataFrame,  
  
# Replace 'path_to_your_file.csv' with the actual file path  
import pandas as pd  
filename = "group01_http_input.csv" # "path_to_your_file.csv"  
messages_df = pd.read_csv(filename)  
messages_df
```

	msg_id	app_protocol	src_app	dst_app	message	timestamp	src_port	dst_port
0	1	HTTP	client_browser	web_server	[PROJECT-GROUP-01] GET /index.html HTTP/1.1	0.015	50100	80
1	2	HTTP	client_browser	web_server	[PROJECT-GROUP-01] GET /style.css HTTP/1.1	0.005	50101	80
2	3	HTTP	client_browser	web_server	[PROJECT-GROUP-01] POST /submit_form HTTP/1.1	0.010	50102	80
3	4	FTP	file_client	file_server	[PROJECT-GROUP-01] USER admin; RETR homework.doc	0.020	40500	21
4	5	DNS	user_pc	dns_server	[PROJECT-GROUP-01] query google.com	0.030	33000	53

Step 2 — Validate the Schema

The notebook will automatically check the CSV header. If a required column is missing, you will see an error message.

- If validation fails, fix your CSV and re-run Step 1.
- If it passes, continue to Step 3.

```
[3]: # TODO: Run the cell to validate the CSV file format  
  
def validate_csv_format(df: pd.DataFrame):  
    expected_columns = ["app_protocol", "src_port", "dst_port", "message", "timestamp"]  
    for col in expected_columns:  
        if col not in df.columns:  
            raise ValueError(f"Missing expected column: {col}")  
  
messages_df['message'] = messages_df['message'].fillna('') # Fill NaN messages with empty strings  
validate_csv_format(messages_df)
```


4. אריזה סופית: איחוד כל השכבות למחרוזת בינארית אחת המייצגת חבילת רשת תקנית.

"איור 2: מימוש פונקציית Checksum לבקרת שגיאות ופונקציית Hexdump להצגת נתונים בינאריים"

▼ Step 3 — Map to TCP/IP Layers (Encapsulation)

This notebook will map each application message to the TCP/IP stack:

- Application → Transport (e.g., TCP/UDP headers)
- Internet (IP headers)
- Link (frame headers/footers)

Just run the cell(s) in this section to see the derived structures.

```
[ ]: # NOTE: Follow the step-by-step markdown cells above. Set paths only where indicated.
import socket, struct, random, time, platform
from typing import Optional

IS_WINDOWS = (platform.system() == 'Windows')
try:
    from scapy.all import IP as SCAPY_IP, TCP as SCAPY_TCP, Raw as SCAPY_Raw, send as scapy_send, get_if_list
    HAVE_SCAPY = True
except Exception as e:
    HAVE_SCAPY = False
    SCAPY_IMPORT_ERR = e
IS_WINDOWS, HAVE_SCAPY
```

```
[ ]: # Function to calculate checksum
def checksum(data: bytes) -> int:
    if len(data) % 2:
        data += b'\0'
    res = sum(struct.unpack('!%dH' % (len(data)//2), data))
    while res >> 16:
        res = (res & 0xFFFF) + (res >> 16)
    return ~res & 0xFFFF

# Helper function to display the data
def hexdump(data: bytes, width: int=16):
    for i in range(0, len(data), width):
        chunk = data[i:i+width]
        hex_bytes = ' '.join(f'{b:02x}' for b in chunk)
        ascii_bytes = ''.join(chr(b) if 32 <= b < 127 else '.' for b in chunk)
        print(f"{i:04x} {hex_bytes:<{width*3}} {ascii_bytes}")
```

```
# Preview packet structure
src_ip = '127.0.0.1'
dst_ip = '127.0.0.1'
src_port = random.randint(1024, 65535)
dst_port = 12345
payload = b'Hello Packet (preview)'
pkt_preview = build_ip_header(src_ip, dst_ip, 20 + len(payload)) + build_tcp_header(src_ip, dst_ip, src_port, dst_port, payload) + payload
hexdump(pkt_preview)
```

Python

```
0000 45 00 00 3e 2b ab 00 00 40 06 51 0d 7f 00 00 01  E..>...@.Q....
0010 7f 00 00 01 8b c1 30 39 dc 70 76 f5 00 00 00 00  .....09.pv....
0020 50 02 ff ff 8e 96 00 00 48 65 6c 6c 6f 20 50 61  P.....Hello Pa
0030 63 6b 65 74 20 28 70 72 65 76 69 65 77 29      cket (preview)
```

ג. תיאור והסבר של תהליך הלכידה

לאחר בניית החבילה בקוד, ביצענו שליחה שלה דרך הרשת הלוגית של המחשב (Loopback).

- הסבר טכני: השתמשנו במחלקה "RawTcpTransport" בגלל מגבלות אבטחה ב Windows-הקוד משתמש בספריית "Scapy" כדי להזריק את החבילה שבנינו ישירות לכרטיס הרשת.

- במקביל, הפעלנו את תוכנת "Wireshark" במצב האזנה על ממשק ה" Loopback " כדי לתפוס את החבילה ברגע שהיא נשלחת.

איור 5: מימוש ה" Transport " לשליחת חבילות " Raw " ב" Windows " וב" Linux " באמצעות " Scapy ".

Cross-Platform Transport

- Linux/macOS: raw sockets (we include the IP header)
- Windows: Scapy + Npcap fallback (raw TCP sockets are blocked by the OS)

```
[ ]: class RawTcpTransport:
    def __init__(self, src_ip: str, dst_ip: str, src_port: int, dst_port: int, iface: Optional[str]=None):
        self.src_ip = src_ip
        self.dst_ip = dst_ip
        self.src_port = src_port
        self.dst_port = dst_port
        self.iface = iface
        self.windows_fallback = IS_WINDOWS
        if not self.windows_fallback:
            self.sock = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_RAW)
        else:
            if not HAVE_SCAPY:
                raise RuntimeError(
                    f'Windows detected but Scapy is not available: {SCAPY_IMPORT_ERR}.\n'
                    "Install with: pip install scapy. Ensure Npcap is installed with loopback support."
                )

    def encapsulate(self, data: bytes, flags: int=0x02) -> bytes:
        tcp = build_tcp_header(self.src_ip, self.dst_ip, self.src_port, self.dst_port, data, flags=flags)
        ip = build_ip_header(self.src_ip, self.dst_ip, len(tcp) + len(data))
        return ip + tcp + data

    def send(self, data: bytes, flags: int=0x02):
        if not self.windows_fallback:
            pkt = self.encapsulate(data, flags=flags)
            self.sock.sendto(pkt, (self.dst_ip, 0))
        else:
            scapy_pkt = SCAPY_IP(src=self.src_ip, dst=self.dst_ip)/SCAPY_TCP(sport=self.src_port, dport=self.dst_port, flags=flags)/SCAPY_Raw(data)
            chosen_iface = self.iface
            if chosen_iface is None and self.dst_ip in ("127.0.0.1", "::1"):
                chosen_iface = "Npcap Loopback Adapter"
            scapy_send(scapy_pkt, verbose=False, iface=chosen_iface)
```

```
[ ]: # find interface name for Windows
if IS_WINDOWS and HAVE_SCAPY:
    try:
        print('\n'.join(get_if_list()))
    except Exception as e:
        print('Could not list interfaces:', e)
```

```
[ ]: # Preview packet structure
src_ip = '127.0.0.1'
dst_ip = '127.0.0.1'
src_port = random.randint(1024, 65535)
dst_port = 12345
payload = b'Hello Packet (preview)'
pkt_preview = build_ip_header(src_ip, dst_ip, 20 + len(payload)) + build_tcp_header(src_ip, dst_ip, src_port, dst_port, payload) + payload
hexdump(pkt_preview)
```

ד. תיאור והסבר של תעבורה שנלקדה ב- Wireshark

בשלב זה אנו מוודאים שהחבילה שנשלחה מהקוד אכן הגיעה ליעדה וזוהתה נכון על ידי כלי ניתוח סטנדרטיים.

- **ממצאים:** ב-Wireshark ניתן לראות חבילת TCP המכילה בתוכה את נתוני ה-HTTP שלנו.

- **אימות נתונים:** ניתן לראות שפורט המקור והיעד ב-Wireshark-תואמים בדיוק למה שהוגדר ב, CSV-ושהטקסט המזהה של הקבוצה מופיע בבירור בחלק ה Data-של החבילה.

"צילום מסך מתוך Wireshark שבו רואים את החבילה שנלקדה"

